

1. Overview

The **List or Table Selector** is a pluggable web widget for Mendix that provides three interchangeable views for selecting one or more objects: a **Dropdown**, an **Inline List**, and a **Data Table**. All three views share the same data source configuration and selection behaviour, allowing developers to switch between them with a single property change while keeping every other setting intact.

The widget supports associations (single reference and reference set), enumerations, and booleans as data sources, making it a versatile replacement for multiple built-in Mendix input widgets.

2. Key Features at a Glance

Feature	Dropdown	List View	Table View
Single selection (Reference)	Yes	Yes	Yes
Multi selection (Reference Set)	Yes	Yes	Yes
Enumeration / Boolean support	Yes	Yes	No
Search / filter	Yes	Yes	Yes
Custom content in options	Yes	Yes	Yes (per column)
Clearable selection	Yes	Yes	Yes
Keyboard navigation	Yes	Yes	Yes
Select All	Yes	Yes	Yes
Pagination	N/A	Page size scroll cap	Paging buttons or virtual scrolling
Column sorting	N/A	N/A	Yes
Column filtering	N/A	N/A	Yes
Column resizing	N/A	N/A	Yes
Column reordering (drag)	N/A	N/A	Yes
Column visibility toggle	N/A	N/A	Yes
Offline capable	Yes	Yes	Yes

3. Data Source Types

3.1 Association

The most powerful data source type. Configure a **Reference** (single selection) or **Reference Set** (multi selection) association, and a separate **Selectable objects** data source to populate the options.

- **Entity:** The association that connects the page object to the selectable entity.
- **Selectable objects:** A database, microflow, or nanoflow data source that retrieves the available options.
- **Caption type:** Choose between an **Attribute** (a string attribute on the selectable entity) or an **Expression** (a Mendix expression evaluated per object) to display each option's label.

3.2 Enumeration

Binds directly to an enumeration attribute. The options are the enumeration values, and the captions are the enumeration's localized labels. Works with both Dropdown and List views.

3.3 Boolean

Binds directly to a boolean attribute, presenting **True** and **False** as the selectable options with their localized captions. Works with both Dropdown and List views.

Note: The Table view requires an Association data source because it reads its column data from the selectable objects. Enumeration and Boolean are not supported for the Table view; Studio Pro will display a validation error if this combination is configured.

4. View Types

4.1 Dropdown View

The classic selector pattern. Options appear in a floating menu below the input field that opens on click or keyboard interaction.

Behaviour:

- A text input lets the user type to filter the options by the configured filter type (Contains, Starts-with, or None).
- A clear button appears when a value is selected and the widget is clearable.
- A dropdown arrow toggles the menu open and closed.
- For multi selection, selected items can be displayed as **plain text** (comma-separated) or as **removable labels** (pill-shaped badges with an X button).
- An optional **Select All** button in the menu header lets the user select or deselect every visible option at once.
- An optional **footer area** can host arbitrary widgets below the option list (e.g., a "Create new" button).
- Keyboard navigation is powered by the downshift library, providing full ARIA-compliant interaction: arrow keys move the highlight, Enter selects, Escape closes the menu.

Unique properties:

Property	Description
Placeholder text	Text shown when no value is selected.
Show selected items as	Text or Labels (multi selection only).
Selection method	Checkbox or Row click (multi selection only).
Show footer	Adds a widget-hosting area below the option list.

4.2 List View

Every option is rendered inline in an always-visible list. There is no menu to open or close; all options are on screen, each with a native checkbox or radio button.

Behaviour:

- Options are rendered as an ordered list of checkboxes (multi selection or when configured) or radio buttons (single selection).
- An optional search box above the list filters the visible options in real time.
- A **Clear Selection** button resets the selection.
- A **Select All** checkbox (multi selection only) toggles every visible option.
- The list can be laid out **vertically** (one option per row) or **horizontally** (multiple columns filling left to right, using CSS Grid).
- A **Page size** setting caps the number of visible options; when the option count exceeds it, the list becomes scrollable. Set to 0 to show every option without scrolling.

Unique properties:

Property	Description
List view type	Vertical (one option per row) or Horizontal (multi-column grid).
Number of columns	How many options appear side by side in a horizontal layout.
Selection type	Radio button or Check box (single selection only; multi always uses checkboxes).
Show search box	Toggle the search input above the list.
Search placeholder	Placeholder text in the search input.
Page size	Number of options visible before the list scrolls. 0 = show all.

4.3 Table View

Options are presented as rows in a fully configurable data table, with developer-defined columns, sorting, filtering, pagination, and selection controls.

Behaviour:

- Each row represents one selectable object. Columns are configured individually with their own data binding, caption, width, alignment, and capabilities.

- **Selection type** can be either **Checkbox** (a leading column with check boxes or radio buttons) or **Container** (the entire row is clickable, and the selected row is highlighted with a configurable background colour or CSS class).
- A **search box** filters across all searchable columns simultaneously.
- **Per-column filters** add an input under each column header for targeted filtering.
- Columns can be **sorted** (click the header; three states cycle through ascending, descending, and unsorted).
- Columns can be **resized** by dragging the header border.
- Columns can be **reordered** by dragging a header to a new position.
- Columns can be **hidden** by the end user via a column selector dropdown. Individual columns can be set to "hidden by default".
- **Pagination** supports classic paging buttons (above, below, or both) or infinite virtual scrolling.
- An **empty list message** area accepts custom widgets to show when no options match.
- **Dynamic row class** and **dynamic cell class** expressions let developers apply conditional styling per row or per cell.
- **Tooltip** expressions can be set per column for hover text.

Unique properties:

Property	Description
Columns	A list of column definitions, each with its own data binding and settings.
Show column filters	Adds per-column filter inputs below the header.
Show column sorting	Enables clickable headers for sorting.
Show column resizing	Enables drag-to-resize on column borders.
Show column reordering	Enables drag-and-drop header reordering.
Show column selector	Lets the end user hide and show columns at runtime.
Selection type	Check box (leading column) or Container (whole-row click).
Selected row colour	CSS colour for the selected row background (Container mode).
Selected row class	Dynamic class expression applied to selected rows.
Page size	Rows per page.
Pagination	Paging buttons or Virtual scrolling.
Position of paging buttons	Below, Above, or Both.
Show paging buttons	Always or Auto (hidden when only one page).
Empty list message	None or Custom (with a widget drop zone).
Dynamic row class	Expression-based class applied to each row.

Column properties (per column):

Property	Description
Show	Attribute, Dynamic text, or Custom content (widgets).
Attribute	The entity attribute to display.
Dynamic text	A text template with attribute substitutions.
Custom content	A widget drop zone for arbitrary content.
Caption	The column header text.
Tooltip	Hover text per cell (expression).
Visible	Expression controlling column visibility.
Can search	Include this column in search box matching.
Can sort	Allow sorting on this column.
Can filter	Show a filter input for this column.
Can resize	Allow drag-to-resize.
Can reorder	Allow drag-to-reposition.
Can hide	Yes, Yes (hidden by default), or No.
Column width	Auto-fill, Auto-fit content, or Manual (fractional units).
Min width	Auto, Set by content, or Manual (px).
Alignment	Left, Center, or Right.
Wrap text	Allow cell text to wrap to multiple lines.
Dynamic cell class	Expression-based class applied to each cell.

5. Selection Modes

5.1 Single Selection (Reference)

Backed by a Mendix **Reference** association. Exactly one option can be selected at a time. Selecting a new option replaces the previous one.

- Dropdown view: standard single-select behaviour.
- List view: renders radio buttons (or checkboxes that behave as single-select).
- Table view: renders radio buttons in the leading column, or makes rows clickable in Container mode.

5.2 Multi Selection (Reference Set)

Backed by a Mendix **Reference Set** association. Multiple options can be selected simultaneously.

- Dropdown view: selected items shown as text or removable labels. Selection method is either Checkbox (items stay in the list) or Row click (selected items are removed from the list).

- List view: renders checkboxes; a Select All control is available.
- Table view: renders checkboxes in the leading column with a three-state Select All in the header, or makes rows clickable in Container mode.

6. Search and Filtering

6.1 Filter Types

All three views share the same filter-type setting:

Filter Type	Behaviour
Contains	An option matches when the typed text appears anywhere in its caption. Results are ranked by relevance (exact match > starts-with > contains > acronym).
Starts-with	An option matches only when the typed text appears at the start of a word in its caption.
None	Filtering is disabled. In the Dropdown view the input becomes read-only. In the List and Table views the search box is hidden.

6.2 Search Behaviour per View

- **Dropdown:** The input field doubles as the search box. Typing filters the options in real time.
- **List:** A dedicated search box above the list filters the options. The search respects the configured filter type.
- **Table:** A dedicated search box matches across all columns marked as searchable. Additionally, per-column filter inputs (when enabled) allow targeted filtering on individual columns using case-insensitive contains matching.

7. Custom Content

The widget supports embedding arbitrary Mendix widgets inside its options, enabling rich visual representations beyond plain text captions.

7.1 Configuration

The **Custom content** property on the General tab controls this feature:

Value	Effect
No	Options display only their text caption.
Yes	Options display custom widgets everywhere: in the dropdown menu, in the list, and in the selected-value label.

List items only	Custom widgets appear only in the option list/menu, while the selected-value area shows the text caption.
------------------------	---

7.2 Custom Content in Table Columns

Each table column can independently show **Attribute**, **Dynamic text**, or **Custom content** (a widget drop zone). This allows mixing text columns with interactive or visual widget columns in the same table.

8. Accessibility

The widget is built with comprehensive accessibility support:

- **ARIA roles and attributes:** The dropdown uses `combobox` and `listbox` roles (via downshift). The list view uses `radiogroup` or `group` roles. The table view uses `grid`, `rowgroup`, `row`, and `gridcell` roles.
- **Keyboard navigation:** All views are fully operable with the keyboard. Arrow keys, Enter, Space, Escape, and Tab work as expected.
- **Screen reader announcements:** Configurable status messages announce the selected value, the number of available options, and navigation instructions.
- **Native form controls:** The list and table views use real `<input type="checkbox">` and `<input type="radio">` elements rather than custom controls, ensuring compatibility with assistive technologies and browser auto-fill.
- **Configurable ARIA labels:** Clear selection, remove value, and select all buttons each have their own translatable ARIA label.
- **Mendix system properties:** The widget supports the standard Label, Conditional visibility, and Editability system properties.

Configurable Accessibility Properties

Property	Description
Aria required	Marks the input as required for screen readers.
Clear selection button	ARIA label for the clear button (also the visible caption in List and Table views).
Remove value button	ARIA label for the remove button on individual multi-selection labels.
Selected value	Status message prefix for announcing the current selection.
Options available	Status message prefix for announcing the option count.
Instructions	Keyboard navigation instructions read by screen readers.

All text properties support **localization** via Mendix's translation system. Default translations are provided for English (en_US) and Dutch (nl_NL).

9. Events

Event	Description
On change action	Triggered when the selection changes (a value is selected, deselected, or cleared).
On enter action	Triggered when the mouse enters the widget area.
On leave action	Triggered when the mouse leaves the widget area.

10. Studio Pro Integration

10.1 Structure Mode Preview

The widget provides a rich structure-mode preview in Studio Pro:

- **Dropdown:** Shows the input field with the data source caption and a dropdown arrow icon. Custom content drop zones are visible when configured.
- **List:** Shows the search box (when enabled) above a bordered container with the data source caption.
- **Table:** Shows the configured column headers as a header row, plus placeholder body rows. Custom-content columns expose their drop zones for widget placement directly in structure mode.

10.2 Property Visibility

Studio Pro intelligently hides properties that are not relevant to the current configuration:

- Dropdown-specific properties are hidden when the view is List or Table.
- List-specific properties are hidden when the view is Dropdown or Table.
- Table-specific properties are hidden when the view is Dropdown or List.
- Caption properties are hidden for the Table view (it reads from columns).
- Column count is hidden when the list layout is Vertical.
- Pagination position and visibility are hidden for virtual scrolling.
- Individual column capabilities (sort, filter, resize, etc.) are hidden when the global toggle is off.

10.3 Validation

Studio Pro displays errors for invalid configurations:

- Table view with Enumeration or Boolean data source.
- Table view without any columns.
- Columns with missing data bindings (attribute, dynamic text, or custom content).
- Negative page size values.
- Zero columns in a horizontal list layout.

11. Advantages of Using This Widget

11.1 Three Views, One Configuration

The most significant advantage is that all three views (Dropdown, List, and Table) share the same data source, association binding, filter settings, and event configuration. Switching from a dropdown to a table is a single property change on the View tab. This eliminates the need to reconfigure data sources, associations, or events when changing the visual presentation.

11.2 Replaces Multiple Built-in Widgets

This single widget can replace the functionality of several built-in Mendix widgets:

Built-in Widget	Equivalent Configuration
Drop-down (Reference Selector)	Dropdown view + Association + Reference
Reference Set Selector	Dropdown view + Association + Reference Set
Check boxes (Input Reference Set Selector)	List view + Checkboxes
Radio buttons	List view + Radio buttons
Data Grid with selection	Table view + Association

Using one widget instead of many reduces the learning curve, standardizes behaviour across the application, and simplifies maintenance.

11.3 No Helper Entity Required for Checkbox Selection

In standard Mendix, building a checkbox list for a reference set typically requires creating a **helper (non-persistent) entity** with a boolean attribute, populating it from a microflow, rendering it in a list view with checkboxes, and then writing a second microflow to translate the boolean values back into the reference set. This pattern adds domain model clutter, microflow plumbing, and maintenance overhead every time a multi-select input is needed.

The List or Table Selector eliminates this pattern entirely. A checkbox list is achieved by:

1. Pointing the widget at a **Reference Set** association.
2. Setting the view to **List view**.

That is all. The widget reads and writes the reference set directly — no helper entity, no boolean attribute, no population microflow, no commit-back microflow. The same applies to the Table and Dropdown views: multi selection works out of the box with a reference set, regardless of which view is active.

This dramatically reduces the number of entities, microflows, and pages needed in a project, and removes an entire category of bugs related to keeping helper objects in sync with the actual association.

11.4 Universal Filter That Works Across All Configured Values

The widget provides a single, consistent search/filter mechanism that operates on the actual caption values displayed to the user — not on a single hardcoded attribute or a separate XPath constraint.

How it works:

- In the **Dropdown** and **List** views, the search box filters on the option caption (the attribute or expression configured as the Caption on the General tab). The filter type (Contains or Starts-with) is set once and applies everywhere.
- In the **Table** view, the search box matches across **every column marked as searchable** simultaneously. If the user types "Amsterdam", it matches whether "Amsterdam" appears in a City column, an Address column, or a Description column. There is no need to pre-select which column to search — the widget searches all of them in a single pass.
- Table **per-column filters** add targeted narrowing on top of the global search, letting the user combine a broad keyword search with column-specific constraints.

This means there is no need to build separate search microflows, maintain filter entities, or wire XPath constraints to input fields. The filter is built into the selector, operates on exactly what the user sees, and respects the ranking semantics of the configured filter type (relevance-ranked Contains or strict Starts-with). Changing the caption attribute or adding a searchable column automatically includes it in the filter — no extra wiring required.

11.5 Rich Table View with Built-in Selection (replaces Data Grid + Listen Widget)

Unlike the standard Mendix Data Grid, the Table view is purpose-built for selection. It combines data presentation with object selection in a single widget, eliminating the need to wire separate selection microflows or listen widgets.

Key table capabilities not available in a basic reference selector:

- **Sortable columns** with three-state toggling (ascending, descending, unsorted).
- **Per-column filters** for targeted narrowing of options.
- **Resizable columns** via pointer drag.
- **Reorderable columns** via drag-and-drop.
- **Hideable columns** with an end-user column selector.
- **Virtual scrolling** for large datasets without page breaks.
- **Container selection** that makes rows clickable with configurable highlight colours and dynamic CSS classes.
- **Custom content columns** that can host any Mendix widget per cell.

11.6 Flexible List View

The List view provides an inline, always-visible option list that avoids the interaction overhead of opening and closing a dropdown menu.

- **Horizontal layout** arranges options in a multi-column CSS Grid, saving vertical space when there are many short-captioned options.
- **Page size** caps the visible options with smooth scrolling, keeping the page layout stable even with hundreds of options.
- **Native checkboxes and radio buttons** work correctly with browser auto-fill, form submission, and assistive technologies.

11.7 Unified Search Across All Views

Every view provides consistent search and filter behaviour based on the same configurable filter type. The Table view goes further with cross-column search and per-column filter inputs, allowing precise narrowing of large datasets without leaving the selector.

11.8 Custom Content Support

Options can contain arbitrary Mendix widgets instead of plain text labels. This enables:

- Product cards with images, prices, and ratings as selectable options.
- User avatars with names and roles.
- Status indicators with coloured badges.
- Any other rich visual representation.

The "List items only" mode displays custom content in the option list but keeps the selected-value display as plain text, which is ideal when the dropdown input needs to remain compact.

11.9 Multi Selection with Ergonomic Controls

Multi selection is a first-class feature across all views:

- **Select All / Deselect All** in the dropdown menu header, the list toolbar, and the table header.
- **Removable labels** in the dropdown view display selected items as pill-shaped badges that can be individually removed with a click.
- **Three-state checkbox** in the table header indicates whether all, some, or no rows are selected.

11.10 Full Accessibility Compliance

The widget meets WCAG accessibility standards with proper ARIA roles, keyboard navigation, screen reader announcements, and native form controls. Accessibility labels are fully translatable.

11.11 Offline Capable

The widget is marked as offline capable, supporting Mendix applications that need to function without a network connection.

11.12 Design-Time Experience

The Studio Pro integration includes:

- Context-sensitive property panels that show only relevant settings.
- Validation errors for invalid configurations caught before runtime.
- Structure-mode previews that show the widget's layout with actual column headers and drop zones.
- Custom captions in the page editor that reflect the configured data source.